

Paralelización del Filtro NL-Means Color para Video

CPU OpenMP vs GPU CUDA Global vs GPU CUDA Shared Memory

Integrantes:

Diego Silva Madariaga

Pablo Ibanez

Profesor: Michael Miranda

Ramo: Computación Paralela y Distribuida

Primer Semestre 2026

Este informe fue desarrollado con asistencia del modelo Claude sonnet 4.6 como herramienta de apoyo para desarrollo de código y redacción. El contenido técnico, experimental y analítico es de autoría de los integrantes.

1. Resumen

El ruido en video digital es un problema en la captura y compresión de imágenes. Este proyecto implementa el algoritmo NL-Means Color (NLM-C) para eliminación de ruido en secuencias de video, evaluando tres estrategias de paralelización

- CPU multi-hilo con OpenMP (16 hilos)
- GPU CUDA con memoria global (DRAM, memoria principal de la GPU)
- GPU CUDA con memoria compartida (SRAM, memoria pequeña y rápida dentro de cada multiprocesador)

El experimento se hizo con un video de un entorno urbano de 1.280x560 a 30 FPS, con ruido Gaussiano artificial de sigma 25. Los experimentos se ejecutaron sobre una NVIDIA RTX 4060 (3.072 CUDA Cores), 16gb de RAM y una CPU Ryzen 5700X

Usando CPU OpenMP como línea base (8.157 segundos / 135 minutos), la versión GPU CUDA alcanza un speedup de 43,2, mientras que la GPU con memoria compartida resultó más lenta con un speedup de 31,7. un resultado contraintuitivo explicado por limitaciones de la arquitectura de la GPU, pues al cargar demasiados datos en la memoria rápida interna de la GPU (21 KB por bloque), el hardware solo puede ejecutar un grupo de hilos a la vez en lugar de varios en paralelo, perdiendo el beneficio que esa memoria debía aportar.

El proyecto demuestra la eficacia de la GPU para NLM-C y evidencia que usar memoria compartida no siempre acelera, cuando el algoritmo requiere cargar demasiados datos por bloque de hilos, puede ocupar toda la memoria rápida disponible y terminar siendo más lento que acceder directamente a la memoria global.

2. Introducción y Motivación

El ruido en señales de video es un fenómeno inevitable en la captura digital, especialmente bajo condiciones de baja iluminación, alta sensibilidad ISO o compresión agresiva. El más frecuente es el Gaussiano aditivo, que degrada la calidad visual y dificulta tareas posteriores como compresión, análisis de movimiento o visión computacional.

El filtro Non-Local Means (NL-Means), propuesto por Buades, Coll y Morel en 2005, es uno de los algoritmos de filtrado de ruido(denoising) más reconocidos por su capacidad de preservar detalles y texturas. A diferencia de filtros locales como Gaussiano o mediana, NL-Means promedia píxeles según la similitud estructural de parches de imagen, explotando la redundancia no-local de las escenas naturales. Su variante para color, NLM-C, extiende la comparación de parches a los tres canales RGB de forma simultánea, obteniendo resultados más coherentes en imágenes y video en color.

La complejidad de NLM-C es $O(N \times W^2 \times P^2)$ por cuadro, donde N es el número de píxeles, W el radio de ventana de búsqueda y P el radio del parche. Para el video de prueba (1.280x560, ventana 21x21, parche 7x7), esto equivale a aproximadamente 1.120 millones de operaciones de punto flotante por cuadro, haciendo inviable una implementación secuencial en tiempo razonable.

Este proyecto implementa NLM-C sobre GPU con CUDA para obtener tiempos prácticos, compara estrategias de memoria (global vs compartida), y cuantifica el speedup real sobre una línea base de CPU paralela con OpenMP.

3. Objetivo General y Objetivos Específicos

Objetivo General

Implementar y evaluar el filtro NLM-C para video en tres versiones paralelas (CPU OpenMP, GPU CUDA global, GPU CUDA shared memory), analizar el rendimiento, speedup y comportamiento de la memoria.

Objetivos Específicos

- Implementar NLM-C con paralelismo CPU usando OpenMP con 16 hilos.
- Implementar NLM-C con paralelismo GPU masivo usando CUDA con memoria global (un hilo por píxel).
- Implementar NLM-C con CUDA usando memoria compartida para reducir accesos a la memoria principal de la GPU.
- Medir tiempos de procesamiento por cuadro, tiempo total y speedup de las tres versiones sobre el mismo video.
- Analizar el impacto del uso de memoria compartida en la ocupación del SM y el rendimiento del kernel.

4. Datos Utilizados

Para los experimentos principales se utilizó un video de entorno urbano (Chicago) al que se le aplico ruido Gaussiano artificial mediante un script (agregar_ruido.py) desarrollado como parte del proyecto. Las características del video son las siguientes.

- Formato MP4, resolución de 1.280 x 560 pixeles, 30 FPS (tasa de cuadros)
- Numero de cuadros: 1.459 (duración aprox. 48,6 segundos)
- Ruido Gaussiano aditivo $\mu=0$, $\sigma=25$, semilla `np.random.seed(42)`
- Tamaño por cuadro: $1.280 \times 560 \times 3 = 2.150.400$ bytes (~2,05 MB/cuadro)

El ruido fue generado con `np.random.normal(0, 25, frame.shape)` y sumado píxel a píxel, recortando a `[0, 255]`. Para la validación funcional del pipeline se utilizó adicionalmente `t80.mp4` (video de combate), aunque los resultados reportados corresponden exclusivamente a `city_ruido_sigma25.mp4`.

5. Línea Base

La línea base corresponde a la implementación CPU con paralelismo OpenMP de 16 hilos. Una ejecución secuencial pura resulta inviable, pues se estima que su duración seria x10 la de OpenMP, superaría las 22 horas, por lo que no fue incluida como referencia medible.

Métricas de la Línea Base (CPU OpenMP, 16 hilos)

- Tiempo total de procesamiento: 8.157.200 ms (8.157 segundos / 135 minutos)
- Tiempo por cuadro: 5.590,95 ms
- Cuadros procesados: 1.459
- Resolución: 1.280 x 560
- Hardware: Ryzen 5700x 16 hilos OpenMP

La paralelización OpenMP distribuye el cálculo usando la directiva "parallel for collapse(2) schedule(dynamic) num_threads(16)". La cláusula collapse(2) expande el doble loop sobre filas y columnas en un único dominio paralelo, y la política dynamic asigna bloques de píxeles dinámicamente entre los 16 hilos para balancear la carga, dado que el tiempo de cómputo por píxel varía según cuantos vecinos caen dentro de los límites del cuadro.

6. Diseño de la Solución

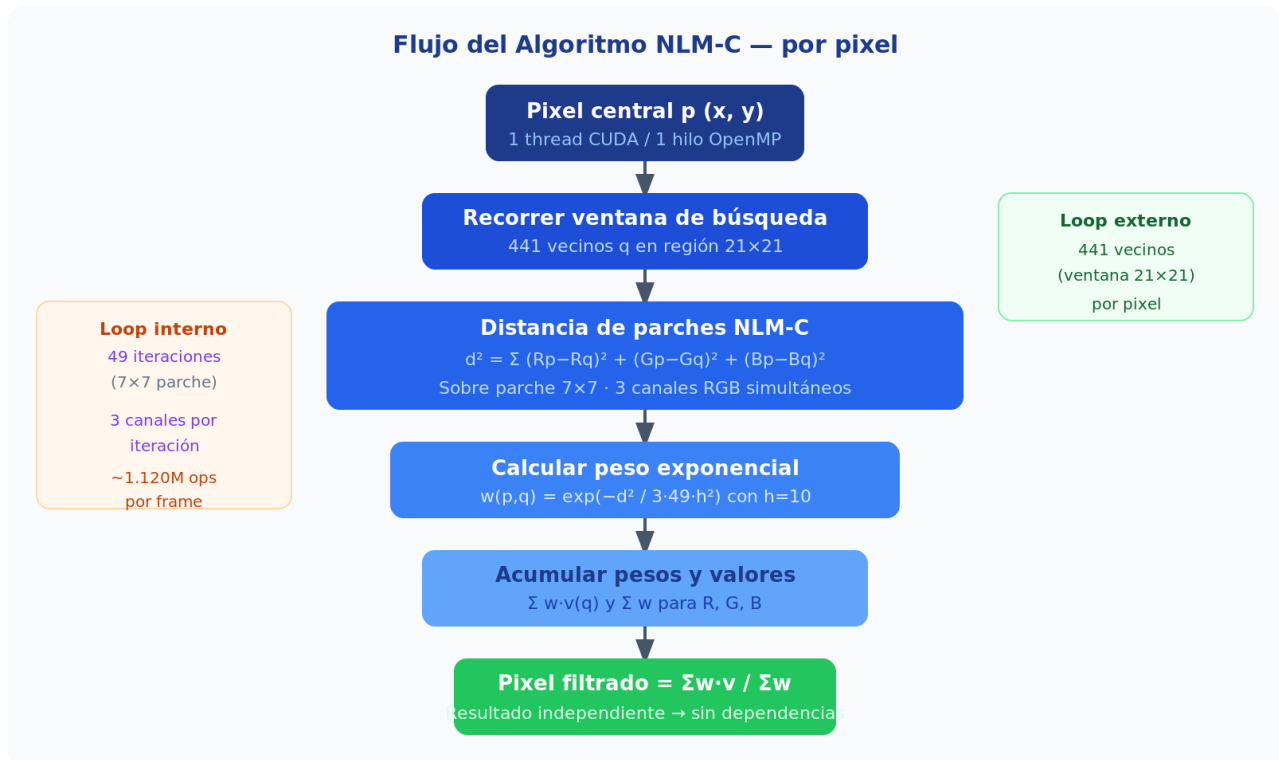


Figura: Flujo del algoritmo NLM-C por píxel. Cada thread procesa un píxel de forma completamente independiente.

Parámetros del Algoritmo

- Parche de tamaño 7 x 7 píxeles
- Ventana de búsqueda de 21 x 21 píxeles (radio mv = 10)
- Parámetro de filtrado h = 10
- Canales: R, G, B procesados conjuntamente en la distancia de parches (NLM-C)

Estrategias de Paralelización

- CPU OpenMP: el doble loop de píxeles se distribuye dinámicamente entre 16 hilos con collapse(2). Cada hilo computa subconjuntos de filas de forma independiente.
- GPU CUDA Global: un hilo por píxel, grid de bloques 16x16. Acceso a memoria global para todos los parches y la ventana de búsqueda.
- GPU CUDA Shared: igual a GPU Global, pero cada bloque de hilos carga colaborativamente una región extendida de píxeles en memoria compartida (3 canales) antes del cómputo, usando __syncthreads() como barrera de sincronización.

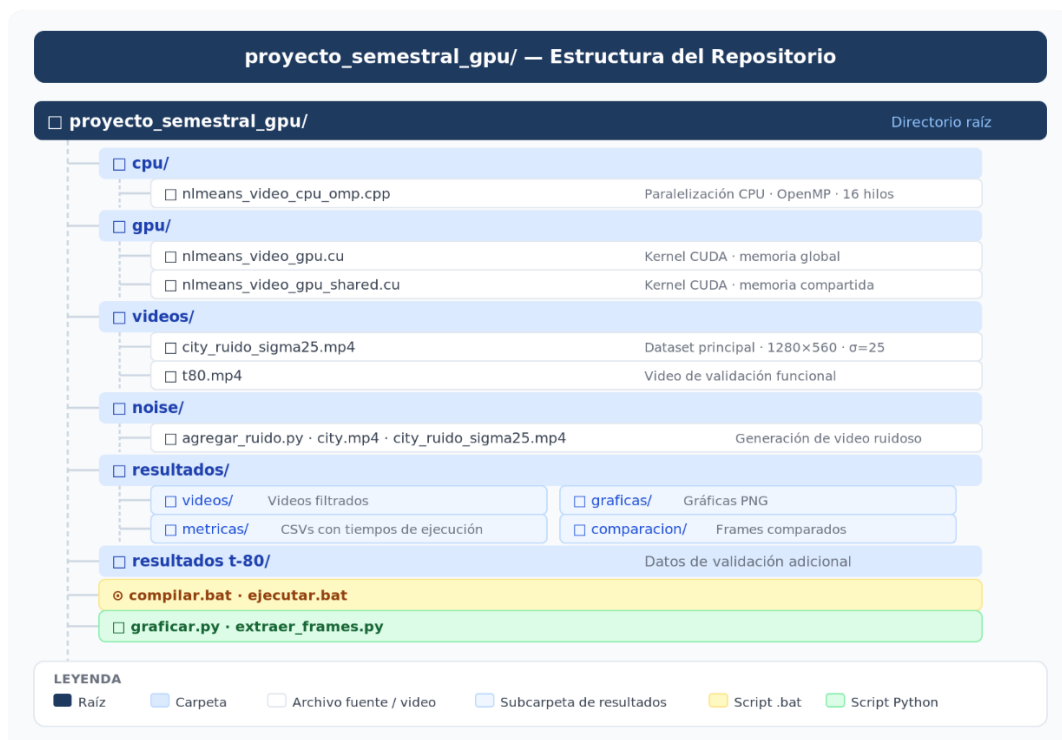


Figura: Comparación de las tres estrategias de paralelización. CPU OpenMP distribuye filas entre 16 hilos; GPU Global asigna 1 thread por pixel; GPU Shared carga una región extendida de píxeles en memoria compartida.

Cálculo de la Región de Memoria Compartida

Para cargar la región de memoria compartida, cada bloque de 16×16 hilos necesitan también los píxeles del borde de búsqueda (radio 13 px a cada lado), resultando en una región de 42×42 píxeles por canal. Con 3 canales RGB y 4 bytes por píxel, esto ocupa 20,7 KB por bloque.

7. Implementación



Estructura del Proyecto

Compilación y Ejecución

El proyecto incluye scripts de automatización para compilación y ejecución, ejecutando `compilar.bat` desde una consola x64 Native Tools, el cual llama internamente al compilador MSVC con optimizaciones openmp para la versión CPU y a `nvcc` con arquitectura `sm_89` para las versiones GPU. La ejecución de las tres versiones se realiza con `ejecutar.bat`, que integra un menú básico de selección de video, lanza CPU, GPU Global y GPU Shared en secuencia y guarda los resultados en la carpeta `resultados`. Los gráficos y comparaciones de frames se generan posteriormente con los scripts `graficar.py` y `extraer_frames.py`.

Decisiones de Implementación

- Memoria GPU pre-alocada: los arreglos de píxeles se reservan una sola vez antes de procesar el video y se reutilizan cuadro a cuadro, evitando el costo de reservar y liberar memoria en cada frame.
- Medición de tiempo exclusiva del kernel: el tiempo GPU se mide solo durante el procesamiento, sin incluir la transferencia de datos entre CPU y GPU, para reflejar el rendimiento real.
- Lectura de archivos de video compatible con Windows: el listado de videos disponibles usa funciones nativas de Windows, sin dependencias externas adicionales.
- Validación de entrada del usuario: si el usuario ingresa un valor no numérico o inválido al seleccionar el video, el programa lo detecta y vuelve a pedir la entrada sin fallar.
- Nombre de salida automático: el video filtrado se guarda con un nombre derivado del video de entrada, evitando que distintas ejecuciones sobrescriban el mismo archivo.

Lógica del Kernel GPU

El kernel CUDA lanza un thread por píxel del frame. Cada thread calcula de forma independiente el píxel filtrado recorriendo todos los vecinos en la ventana de búsqueda (441 posiciones), acumulando pesos exponenciales proporcionales a la similitud de parches en los 3 canales RGB. Al finalizar divide la suma ponderada por la suma de pesos, obteniendo el píxel de salida sin ninguna dependencia entre threads. Para la versión con memoria compartida, antes del cómputo cada bloque carga colaborativamente una región extendida de 42x42 píxeles por canal en memoria compartida, que luego es accedida localmente durante el cálculo. El código fuente completo se encuentra en los archivos `nlmeans_video_gpu.cu` y `nlmeans_video_gpu_shared.cu` del repositorio.

8. Diseño Experimental

Entorno de Hardware y Software

Componente	Especificación
GPU	NVIDIA RTX 4060 — 3.072 CUDA cores, 8 GB GDDR6
Shared Mem/SM	32 KB por defecto (hasta 100 KB)
CPU / Hilos OMP	16 hilos OpenMP
Sistema Operativo	Windows 11 x64
CUDA Toolkit	12.x (<code>nvcc -arch=sm_89</code>)
Compilador C++	MSVC x64 Native Tools (/O2 /openmp)
OpenCV	4.12.0 (instalada con <code>vcpkg</code>)

Variables Controladas

- Video de entrada: city_ruido_sigma25.mp4 (fijo para las 3 versiones, ruido seed 42)
- Parámetros NLM-C: parche=7x7, ventana=21x21, h=10,0 (idénticos en las 3 implementaciones)
- Bloques CUDA: 16x16 threads por bloque (block_size=16)
- Cuadros procesados: 1.459 (video completo)

Variables Medidas

- Tiempo de kernel GPU medido con CUDA Events, excluye la transferencia de datos entre CPU y GPU
- Tiempo total CPU: medido con std::chrono, incluye el costo de gestión de hilos OpenMP.
- Speedup: tiempo CPU / tiempo GPU, indicando cuántas veces más rápida es cada versión GPU respecto a la línea base.
- Tiempo promedio por cuadro: tiempo total / número de frames procesados.

Protocolo de Ejecución

Las tres versiones se ejecutaron secuencialmente sobre el mismo video usando el script ejecutar.bat. Cada versión procesa el video completo y genera automáticamente un archivo CSV en la carpeta de resultados con las métricas.

9. Resultados

Versión	Tiempo Total (s)	Tiempo/Frame (ms)	Speedup vs OMP
CPU OpenMP (16 hilos)	8.157,2	5.590,95	1x (baseline)
GPU CUDA Global	188,78	129,39	43,2x
GPU CUDA Shared Memory	257,56	176,53	31,7x

Tabla 1. Tiempos de procesamiento y speedup de las 3 versiones sobre city_ruido_sigma25.mp4 (1.280x560, 1459 frames, sigma=25).

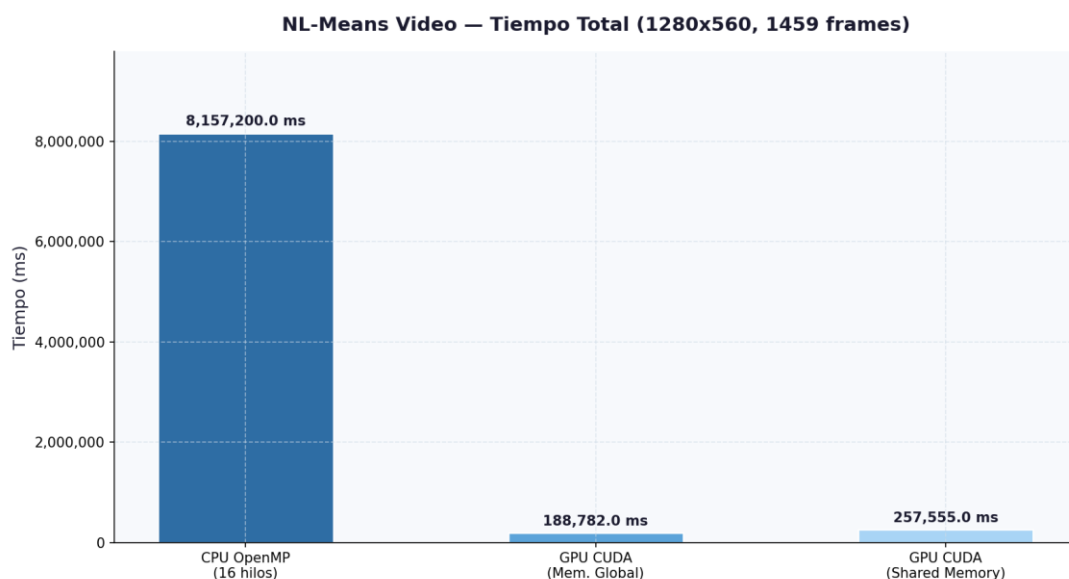


Figura 1. Tiempo total de procesamiento del video completo. CPU OpenMP: 8.157 s vs. GPU Global: 188 s.

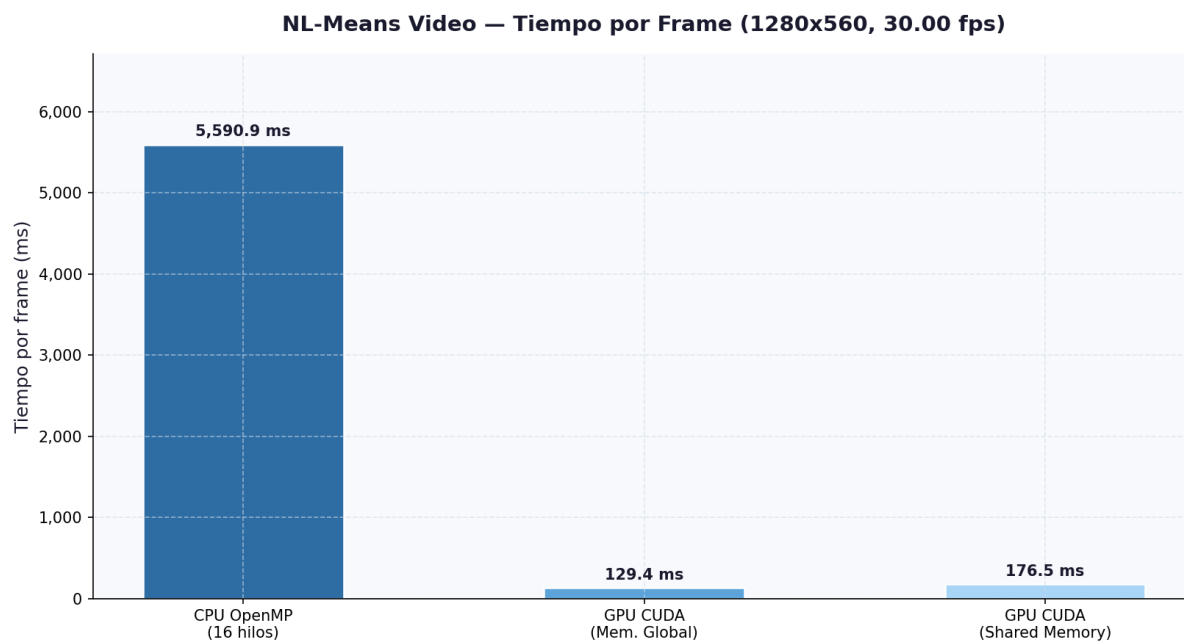


Figura 2. Tiempo promedio por cuadro. GPU Global: 129 ms/frame; CPU OpenMP: 5.591 ms/frame.

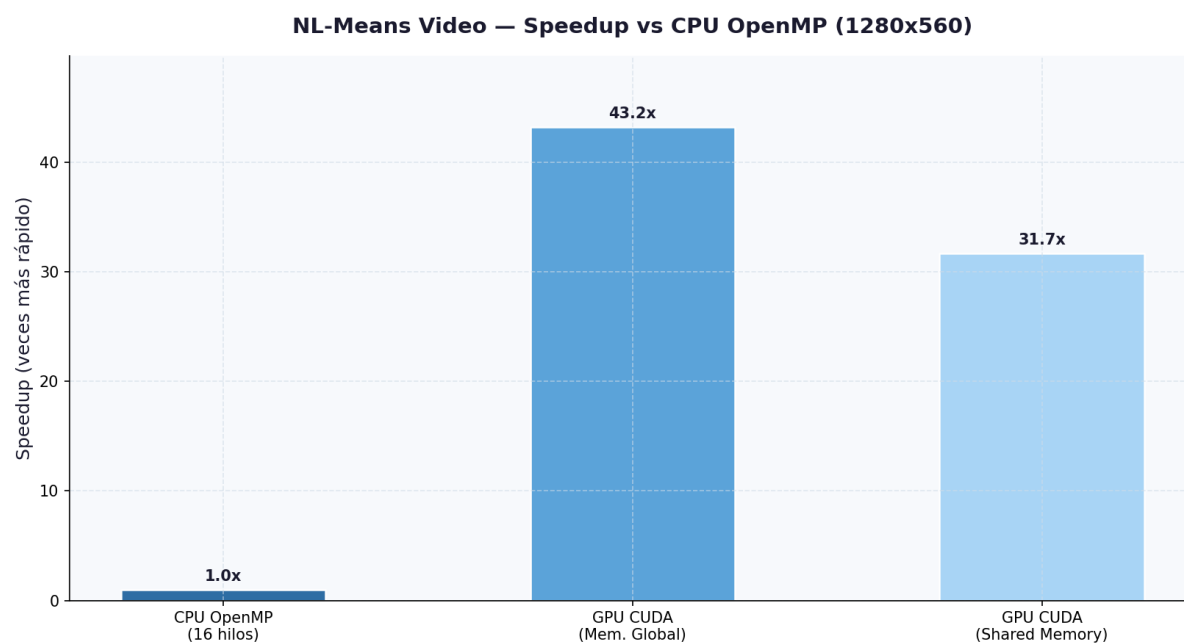


Figura 3. Speedup relativo a CPU OpenMP. GPU Global: 43,2x; GPU Shared: 31,7x.

Calidad Visual / Comparación de Frames

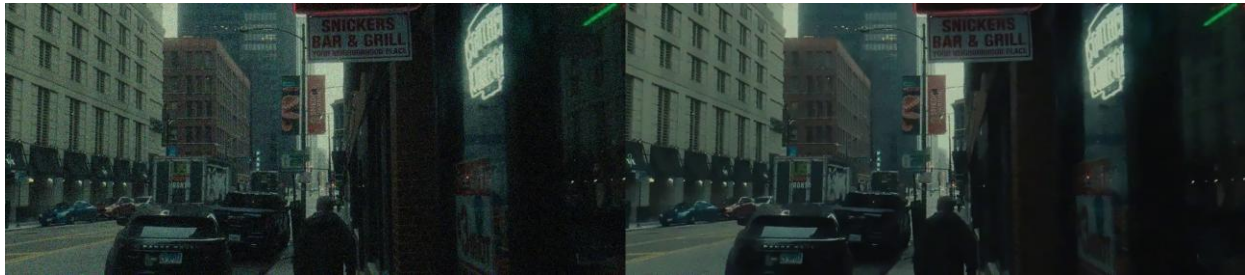


Figura 5. Frame 332: original ruidoso (izq.) vs. filtrado NLM-C GPU (der.).



Figura 6. Frame 510: zona con edificios. El NLM-C preserva bordes y elimina ruido Gaussiano.

10. Análisis y Discusión

CPU OpenMP

La versión CPU con OpenMP y 16 hilos procesó el video completo en casi 136 minutos (8.157 segundos). Si bien el uso de `collapse(2)` y `schedule(dynamic)` distribuye correctamente el trabajo entre los hilos y explota los núcleos disponibles, el rendimiento está acotado por el ancho de banda de memoria del CPU y por la naturaleza del algoritmo, para cada píxel de salida NLM-C accede a una ventana de búsqueda de 21×21 posiciones y evalúa parches de 7×7 , lo que implica una cantidad de operaciones de punto flotante y accesos a memoria de alta intensidad que el CPU ejecuta de forma secuencial por hilo. El rendimiento de la CPU no puede competir con el paralelismo masivo de la GPU para este tipo de carga de trabajo.

GPU CUDA Global. El Mejor Resultado del Proyecto

La versión GPU con memoria global tardó solo 188 segundos con un speedup de 43,2x sobre la línea base OpenMP. La clave está en la correspondencia natural entre el algoritmo y la arquitectura CUDA, al no existir dependencias entre los píxeles de salida, se puede asignar exactamente un thread por píxel, lanzando 3.072 threads en paralelo por frame. La memoria global es accedida de forma eficiente cuando los hilos de un mismo grupo acceden a posiciones contiguas de memoria simultáneamente. Al no usar memoria compartida, el procesador puede mantener entre 8 y 16 grupos de hilos activos al mismo tiempo, ocultando las esperas de acceso a memoria con trabajo útil de otros hilos. Este resultado confirma que NLM-C es un algoritmo naturalmente paralelizable a nivel de píxel de salida.

GPU CUDA Shared. El Hallazgo Contraintuitivo

La versión con shared memory resultó un 36% más lenta que la versión global, con un speedup de 31,7x. Este resultado es el más relevante del proyecto porque contradice la intuición inicial, cargar datos en shared memory debería reducir accesos a memoria global y acelerar el kernel. Sin embargo, el costo de ese beneficio es el consumo de recursos por bloque. Cada bloque carga una región extendida de 42×42 píxeles en los 3 canales RGB, ocupando 20,7 KB de los 32 KB de shared memory disponibles por SM en la RTX 4060.

Esto limita el planificador a un único bloque activo por multiprocesador, eliminando exactamente el mecanismo de ocultamiento de latencia que hace eficiente la versión global. A esto se suman los accesos a memoria global en los píxeles de borde de la ventana de búsqueda que quedan fuera de la región cargada, y la división de ejecución entre hilos que leen desde memoria compartida y los que deben ir a memoria global. El resultado neto es una versión más compleja y lenta.

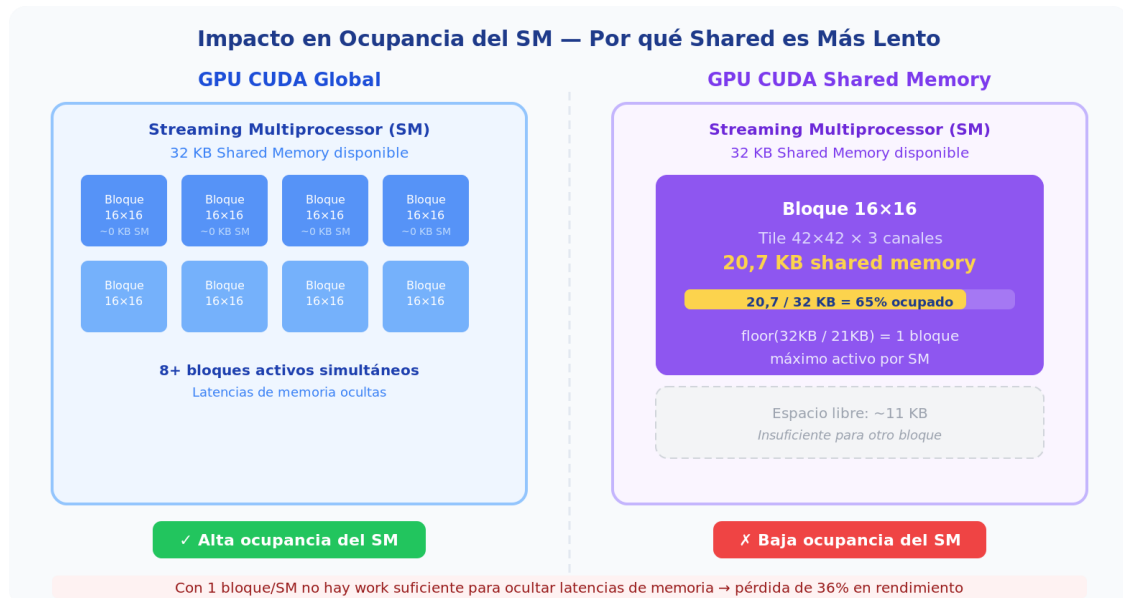


Figura 7. Comparación de ocupación: GPU Global mantiene 8+ bloques activos por SM; GPU Shared queda limitada a 1 bloque por SM debido al consumo de 20,7 KB de shared memory, eliminando el ocultamiento de latencias.

Calidad Visual: Las Tres Versiones Son Equivalentes

Desde el punto de vista visual, las tres implementaciones producen el mismo resultado, el algoritmo NLM-C es exacto y no aproximado, por lo que OpenMP, GPU Global y GPU Shared generan el mismo frame filtrado dado el mismo video de entrada y los mismos parámetros.

Las comparaciones de frames mostradas en la sección de resultados muestran que el filtro elimina eficazmente el ruido en zonas de cielo y superficies uniformes, mientras preserva bordes nítidos en fachadas de edificios, señales y elementos con detalle fino. Esto se debe a que NLM-C pondera píxeles por similitud de contenido en lugar de distancia espacial, lo que le permite suavizar sin difuminar bordes. La diferencia entre las tres versiones es exclusivamente de rendimiento, no de calidad.

Relevancia para el Ramo de Computación Paralela y Distribuida

Este proyecto puso en práctica conceptos fundamentales del ramo. La comparación entre las tres versiones ilustra directamente los distintos niveles de paralelismo, hilos de CPU con memoria del sistema (OpenMP), miles de hilos GPU con acceso a memoria principal de la GPU (Global), y carga colaborativa en memoria rápida interna de la GPU (Shared).

El speedup de 43,2x sobre 16 hilos CPU demuestra que la escala de paralelismo importa, pasar de 16 a 3.000 hilos de ejecución produce un salto gigante en el rendimiento. El resultado de shared memory ilustra además que un kernel más complejo puede ser más lento si reduce la cantidad de trabajo que el hardware puede hacer en paralelo. El proyecto entregó evidencia cuantitativa del paralelismo, niveles de memoria y complejidad de implementación.

Limitaciones del Estudio

- La línea base es OpenMP no es secuencial, el speedup real sobre el algoritmo secuencial sería mayor.
- El tiempo GPU reportado excluye transferencias H2D/D2H, que en un pipeline real representan overhead adicional.
- El video tiene resolución no estándar (1.280x560), lo que genera bloques incompletos en los bordes del frame.

11. Conclusiones

El proyecto implementó exitosamente el filtro NLM-C en tres versiones paralelas, confirmando que el algoritmo es naturalmente apto para la GPU, al no existir dependencias entre los píxeles de salida, miles de hilos pueden trabajar de forma completamente independiente. La versión GPU con memoria global alcanzó un speedup de 43,2x sobre la línea base CPU con 16 hilos, reduciendo el tiempo por frame de 5,6 segundos a 129 milisegundos. Este resultado demuestra que, cuando el algoritmo lo permite, la escala de paralelismo de la GPU representa una ventaja real y contundente.

El hallazgo más relevante fue que la versión con memoria compartida resultó más lenta, no más rápida. Esto porque cargar una región extendida de píxeles en la memoria rápida interna de la GPU consume tanto espacio que el hardware queda limitado a procesar un solo grupo de hilos a la vez, perdiendo la capacidad de ocultar tiempos de espera con trabajo paralelo. Este resultado enseña que optimizar para GPU requiere considerar no solo la velocidad de la memoria, sino cuántos recursos consume cada grupo de hilos y cómo eso afecta el paralelismo real.

Desde el punto de vista visual, las tres versiones producen resultados idénticos, el filtro elimina eficazmente el ruido preservando bordes y detalles, lo que confirma su vigencia como algoritmo de referencia. En definitiva, el proyecto no solo entregó implementaciones funcionales con mediciones sólidas, sino una comprensión práctica de cuándo y por qué la GPU representa una ventaja, y de las condiciones bajo las cuales una optimización puede volverse contraproducente.

12. Referencias

-
- [1] Buades, A., Coll, B., & Morel, J. M. (2005). A non-local algorithm for image denoising. IEEE CVPR 2005, vol. 2, pp. 60-65.
 - [2] NVIDIA Corporation. (2024). CUDA C++ Programming Guide v12.x. <https://docs.nvidia.com/cuda/>
 - [3] OpenCV. (2024). OpenCV 4.12.0 Documentation. <https://docs.opencv.org/4.12.0/>
 - [4] OpenMP Architecture Review Board. (2021). OpenMP API v5.2. <https://www.openmp.org/specifications/>
 - [5] Anthropic. (2025). Claude AI (claude-sonnet-4-6). Herramienta de asistencia en desarrollo y redacción. <https://www.anthropic.com/>

13. Anexos

A.1 Instrucciones de Compilación y Ejecución

- Requisito: NVIDIA CUDA Toolkit 12.x (nvcc en PATH)
- Requisito: MSVC x64 Native Tools Command Prompt (cl.exe)
- Requisito: OpenCV 4.12.0 vía vcpkg (C:\vcpkg\installed\x64-windows)
- Paso 1: Abrir "x64 Native Tools Command Prompt for VS 2022".
- Paso 2: Navegar a la carpeta proyecto_semestral_gpu.
- Paso 3: Ejecutar compilar.bat para compilar las 3 versiones.
- Paso 4: Ejecutar ejecutar.bat para procesar un video (menú básico).
- Paso 5: Ejecutar graficar.py para generar las gráficas de métricas.
- Paso 6: Ejecutar extraer_frames.py para generar las comparaciones de frames.

A.2 Comparaciones de Frames Adicionales



Figura A.1. Frame 145 — inicio del video. Eliminación de ruido con preservación de texturas.



Figura A.2. Frame 672. Nitidez de edificios preservada.

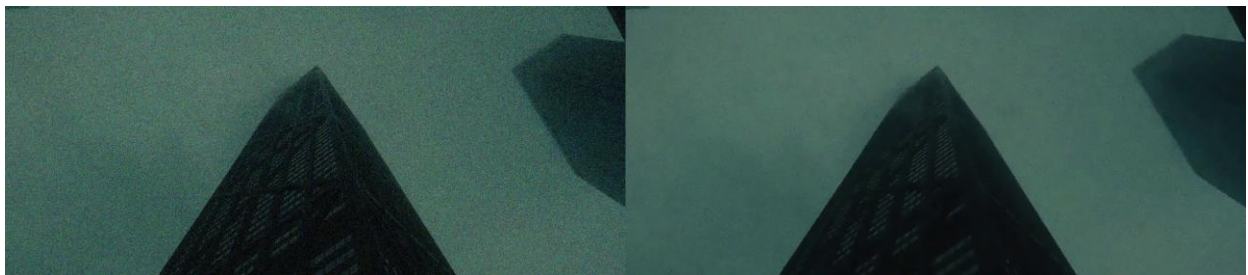


Figura A.3. Frame 1070 . Denoising efectivo en zonas dinámicas.